Part 1: Foundations

Chapter 1: LLMs and the Need for RAG

This Chapter Covers

- The limits of LLMs and the need for RAG
- The RAG basics
- Popular use cases of RAG

Summary

- RAG enhances the memory of LLMs by providing access to external information.
- LLMs are next-word (or token) prediction models trained on massive amounts of text data to generate human-like text.
- LLMs face challenges of having a knowledge cut-off date and being trained only on public data. They are also prone to generating factually incorrect information (i.e., hallucinating).
- RAG overcomes the LLM limitations by incorporating non-parametric memory and increases context awareness and reliability of responses.
- Popular use cases of RAG include search engines, document question-answering systems, conversational agents, personalized content generation, virtual assistants, and so forth.

Chapter 2: RAG Systems and their Designs

This Chapter Covers

- The concept and design of RAG systems
- An overview of the indexing pipeline
- An overview of the generation pipeline
- An initial look at RAG evaluation
- A high-level look at the RAG operations stack

RAGOps Stack

1. Data layer: Tools and platforms used to process and store data in the form of embeddings
2. Model layer: Providers of proprietary or open source LLMs
3. Prompt layer: Tools offering maintenance and evaluation of prompts
4. Evaluation layer: Tools and frameworks providing evaluation metrics for RAG
5. App orchestration: Frameworks that facilitate invocation of different components of the system
6. Deployment layer: Cloud providers and platforms for deploying RAG apps
7. Application layer: Hosting services for RAG apps
8. Monitoring layer: Platforms offering continuous monitoring of RAG apps

Summary

- A RAG-enabled system consists of two main pipelines: the indexing and the generation pipeline.
- The indexing pipeline is responsible for creating and maintaining the knowledge base, which involves data loading, text splitting, data conversion (embeddings), and data storage in a vector database.
- The generation pipeline manages real-time interactions by retrieving information, augmenting queries, and generating responses using an LLM.
- Evaluation and monitoring are crucial components for the assessment of system performance, covering the relevance between the retrieved information and query, the final response and retrieved information, and the final response and the original query.
- The service infrastructure for RAG systems includes layers for data, models, prompts, evaluation, app orchestration, deployment, application hosting, and monitoring.
- Additional components such as caching, guardrails, and security measures are often employed to improve performance, ensure compliance, and address potential threats.

Part 2: Creating RAG systems

Chapter 3: Indexing pipeline: Creating a Knowledge Base for RAG

This Chapter Covers

- Data loading
- Text splitting or chunking
- Converting text to embeddings
- Storing embeddings in vector databases
- Examples in Python using LangChain

Data Loading

Three main activities

- Connecting to the source
- Extracting files
- Parsing text

LangChain is an open source framework developed by Harrison Chase and launched in October 2022. It was written in Python and JavaScript and designed for building applications using LLMs.

Data Loading Component Steps

1. Connect to the source of data.
2. Extract text from the file.
3. Review and update metadata information.
4. Clean or transform the data.

Data Splitting (chunking)

The Chunking Process can be divided into three steps:

1. Divide the longer text into compact, meaningful units (e.g., sentences or paragraphs).

2. Merge the smaller units into larger chunks until a specific size is achieved. After this chunk is treated as an independent segment of text.

3. When creating a new chunk, include a part of the previous chunk at the start of the new chunk. This overlap is necessary to maintain contextual continuity. This process is also known as “small to big” chunking.

Chunking methods

- **Fixed Size Chunking**
    - Split by character
    - Recursively split by character
    - Split by token method (Tiktoken)

- **Specialized Chunking**
    - Split based on structure
    - HTML, Markdown, JSON, or even computer code
    - No longer limited by a fixed width
    - Also called **Adaptive Chunking**

- **Semantic Chunking**
    - Uses embedding similarity to detect topic changes

- **Agentic Chunking**
    - Uses an LLM/agent to determine meaning and structure of a document.

Choosing a Chunking Strategy

- Nature of the content
    - PDF vs HTML, etc...

- Expected length and complexity of user query
    - Short chunks poorly match long queries
- Application and use case requirements
    - Precise answer vs summary
- Embeddings model
    - Certain embeddings models perform better with chunks of specific sizes

Data Conversion: Embeddings

- **Word2Vec**: Word2Vec is a shallow-neural-network-based model for learning word embeddings, developed by researchers at Google. It is one of the earliest embedding techniques.
- **GloVe**: Global Vectors for Word Representations is an unsupervised learning technique developed by researchers at Stanford University.
- **FastText**: FastText is an extension of Word2Vec developed by Facebook AI Research. It is particularly useful for handling misspellings and rare words.
- **ELMo**: Embeddings from Language Models was developed by researchers at Allen Institute for AI. ELMo embeddings have been shown to improve performance on question answering and sentiment analysis tasks.
- **BERT**: Bidirectional Encoder Representations from Transformers, developed by researchers at Google, is a transformers-architecture-based model. It provides contextualized word embeddings by considering bidirectional context, achieving state-of-the-art performance on various NLP tasks.

Embeddings Use Cases

- **Text search**: Searching through the knowledge base for the right document chunk is a key component of RAG systems. Embeddings are used to calculate similarity between the user query and the stored documents.
- **Clustering**: Categorizing similar data together to find themes and groups in the data can result in valuable insights. Embeddings are used to group similar pieces of text together to find out, for example, the common themes in customer reviews.
- **Machine learning**—Advanced machine learning techniques can be used for different problems such as classification and regression. To convert text data into numerical features, embeddings prove to be a valuable technique.
- **Recommendation engines**—Shorter distances between product features mean greater similarity. Using embeddings for product and user features can be used to recommend similar products.

How to choose embeddings?

- Choose based on use case.
- The MTEB leaderboard scores embeddings models across seven use cases:
    - classification,
    - clustering
    - pair classification
    - reranking
    - retrieval
    - semantic text similarity
    - summarization.

Storage: Vector Databases

What are vector databases?

- Vector Databases are built to handle high-dimensional vectors. These databases specialize in indexing and storing vector embeddings for fast semantic search and retrieval.

- Modern vector databases offer traditional features such as scalability, security, multi-tenancy, versioning and management

- Vector Databases are unique in offering similarity searches based on Euclidean distance or cosine similarity.

- They also employ specialized indexing techniques.

Types of vector databases

- **Vector Indexes**: These are libraries that focus on the core features of indexing and search.
    - Facebook AI Similarity Search (FAISS)
    - Non-Metric Space Library (NMSLIB)
    - Approximate Nearest Neighbors Oh Yeah (ANNOY)
    - Scalable Nearest Neighbors (ScaNN)

- **Specialized Vector DBs**: These databases focus on the core feature of high-dimensional vector support, indexing, search, and retrieval such as vector indexes, but also offer database features such as data management, extensibility, security, scalability, non-vector data support, and similar. Examples of specialized vector DBs:
    - Pinecone
    - ChromaDB
    - Milvus
    - Qdrant
    - Weaviate
    - Vald
    - LanceDB
    - Vespa
    - Marqo.

- **Search Platforms**: Solr, Elastic Search, Open Search, and Apache Lucene are traditional text search platforms and engines built for full text search. They have now added vector similarity search capabilities to their existing search capabilities.

- Vector Capabilities for SQL DDs: —Azure SQL, Postgres SQL(pgvector), SingleStore, and CloudSQL are traditional SQL databases that have now added vector data-handling capabilities.

- Vector capabilities for NoSQL databases —Like SQL DBs, NoSQL DBs such as MongoDB have also added vector search capabilities

- Graph databases with vector capabilities —Graph DBs such as Neo4j, have also opened new possibilities by adding vector capabilities

Choosing a Vector Database

- Accuracy vs. speed
- Flexibility vs. performance
- Local vs. cloud storage
- Direct access vs. API
- Simplicity vs. advanced features
- Cost

Summary

DATA LOADING

- The process of sourcing data from its original location is called data loading, and it includes the following four steps: connecting to the source, extracting and parsing text, reviewing and updating metadata, and cleaning and transforming data.
- Loading documents from a list of sources may turn out to be a complicated process. Make sure to plan for all the sources and loaders in advance. A variety of data loaders from LangChain can be used.
- Breaking down long pieces of text into manageable sizes is called data splitting or chunking.
- Chunking addresses context window limits of LLMs, mitigates the lost-in-the-middle problem for long prompts, and enables easier search and retrieval.
- The chunking process involves dividing longer texts into small units, merging small units into chunks, and including an overlap between chunks to preserve contextual continuity.
- Chunking can be fixed size, specialized (or adaptive), or semantic. Newer chunking methods are constantly being introduced.
- Your choice of the chunking strategy should be based on the nature of the content, expected length and complexity of user query, application use case, and the embeddings model being used.
- A chunking strategy can include multiple methods.

DATA CONVERSION

- For processing, text needs to be converted into a numerical format.
- Embeddings are vector representations of data (words, sentences, documents, etc.).
- The goal of an embedding algorithm is to position similar data points close to each other in a vector space.
- Several pre-trained, open source and proprietary, embedding models are available for use.
- Embeddings models enable similarity search. Embeddings can be used for text search, clustering, ML models, and recommendation engines.
- The choice of embeddings is largely based on the use case and the costimplications.
- Vector databases are designed to efficiently store and retrieve high-dimensional vector data such as embeddings.
- Vector databases provide similarity searches based on distance metrics such as cosine similarity.
- Apart from the similarity search, vector databases offer traditional services such as scalability, security, versioning, and the like.
- Vector capabilities can be offered by standalone vector indexes, specialized vector databases, or legacy offerings such as search platforms, SQL, and NoSQL databases with added vector capabilities.
- Accuracy, speed, flexibility, storage, performance, simplicity, access, and cost are some of the factors that can influence the choice of a vector database.

Chapter 4: Generation pipeline: Generating Contextual LLM Responses

This chapter covers

- Retrievers and retrieval methodologies
- Augmentation using prompt engineering techniques
- Generation using LLMs
- Basic implementation of the RAG pipeline in Python

Generation pipeline overview

The generation pipeline involves three processes:

- Retrieval
- Augmentation
- Generation.

The retrieval process is responsible for fetching the information relevant to the user query from the knowledge base. Augmentation is the process of combining the fetched information with the user query. Generation is the last step, in which the LLM generates a response based on the augmented prompt

Retrieval

- Retrieval is a search problem to find the documents that match the input query

Retrieval Methods

- **TD-IDF**

TD(t,D) = Number times t appears in d / tototal terms in d

IDF(t, D) = log(||D|| / number of d in D that contain t)

TD-IDF = TF(t,d) x IDF(t, D)

- **BEST MATCH 25 (BM25)**
    - Improves on TD-IDF by taking into account length of documents

Static Word Embeddings

- Word2Vec & GloVE provide a richer representation
- Capture nuances that TDF-IF and BM25 cannot
- Still lack full contextual understanding
- Limited in handling polysemy

Contextual embeddings

Produce high-dimensional, context-aware representations

- Based on transformers, capture deep semantic meanings and relationships
- Contextual embeddings are the most widely used retrievers in RAG

Other Retrieval Methods

- Learned Sparse Retrieval
- Dense Retrieval
- Hybrid retrieval —Combines sparse and dense methods for balanced efficiency
- Cross-encoder retrieval—Directly compares query-document pairs using trans-
- Graph-based retrieval —Uses graph structures to model relationships between doc-
- Quantum-inspired retrieval—Applies quantum computing principles to IR
- Neural IR models —Encompass various neural network-based approaches to infor-

Augmentation

- Largely falls under the discipline of prompt engineering
- Goal is to construct prompts to achieve accuracy and relevance in responses

RAG prompt engineering techniques

Contextual prompting Controlled generation prompting Few-shot prompting Chain of thought prompting

Other Advanced Prompting Techniques

- **Self-consistency**: While CoT uses a single reasoning chain in CoT prompting, self-consistency aims to sample multiple diverse reasoning paths and use their respective generations to arrive at the most consistent answer.

- **Generated knowledge prompting**: This technique explores the idea of prompt- based knowledge generation by dynamically constructing relevant knowledge chains, using models' latent knowledge to strengthen reasoning.

- **Tree-of-thoughts prompting**: This technique maintains an explorable tree structure of coherent intermediate thought steps aimed at solving problems.

- **Automatic reasoning and tool use (ART)**: —The ART framework automatically inter- leaves model generations with tool use for complex reasoning tasks. ART employs demonstrations to decompose problems and integrate tools without task-specific scripting.

- **Automatic prompt engineer (APE)**: The APE framework automatically generates and selects optimal instructions to guide models. It uses an LLM to synthesize candidate prompt solutions for a task based on output demonstrations.

- **Active prompt**: Active-prompt improves CoT methods by dynamically adapting language models to task-specific prompts through a process involving query, uncertainty analysis, human annotation, and enhanced inference.

- **ReAct prompting**: ReAct integrates LLMs for concurrent reasoning traces and task-specific actions, improving performance by interacting with external tools for information retrieval. When combined with CoT, it optimally utilizes internal knowledge and external information, enhancing the interpretability and trust- worthiness of LLMs.

- **Recursive prompting**: Recursive prompting breaks down complex problems into subproblems, solving them by sequentially using prompts. This method aids compositional generalization in tasks such as math problems or question answering, with the model building on solutions from previous steps.

Generation

The generation step relies completely on the LLM

Categorization of LLMs and Suitability for RAG

Three important themes:

1. How they have been trained
2. How they can be accessed
3. Their size

Original vs. fine-tuned models

- Foundation Models
- Supervised fine-tuning (SFT)

    - SFT adjusts the foundation model weights for specific tasks.

    - SFT is a classification mode-training process

    - SFT Considerations

        - Domain specificity
        - Retrieval integration
        - Deployment speed
        - Customization of responses
        - Resource efficiency
        - Ethical alignment

Open source vs. proprietary models

- Customization
- Ease of use
- Deployment flexibility
- Cost

Model sizes

- Resource constraints
- Reasoning capability
- Deployment options
- Context handling

Summary

Retrieval

- Retrieval is the process of finding relevant information from the knowledge base based on a user query. It is a search problem to match documents with input queries.

- The popular retrieval methods for RAG include

    – TF-IDF (Term Frequency-Inverse Document Frequency) —Statistical measure of word importance in a document relative to a corpus. It can be implemented using LangChain's TFIDFRetriever.

    – BM25 (Best Match 25)—Advanced probabilistic model, an improvement over TF-IDF. It adjusts for document length and can be implemented using Lang Chain's BM25Retriever.

– Static word embeddings —Represent words as dense vectors (e.g., Word2Vec, GloVe) and capture semantic relationships but lack full contextual understanding.

– Contextual embeddings—Produced by models like BERT or OpenAI's text embeddings. They provide context-aware representations and are most widely used in RAG, despite being computationally intensive.

– Advanced retrieval methods —They include learned sparse retrieval, dense retrieval, hybrid retrieval, cross-encoder retrieval, graph-based retrieval, quantum-inspired retrieval, and neural IR models.

- Most advanced implementations will include a hybrid approach.

- Vector stores and databases (e.g., FAISS, PineCone, Milvus, Weaviate), cloud provider solutions (e.g., Amazon Kendra, Azure AI Search, Google Vertex AI Search), and web information resources (e.g., Wikipedia, Arxiv, AskNews) are some of the popular retriever integrations provided by LangChain.

- The choice of retriever depends on factors such as accuracy, speed, and compati- bility with the indexing method.

Augmentation

- Augmentation combines the user query with retrieved information to create a prompt for the LLM.

- Prompt engineering is crucial for effective augmentation, aiming for accuracy and relevance in LLM responses.

- Key prompt engineering techniques for RAG include

  – Contextual prompting—Adding retrieved information with instructions to focus on the provided context. – Controlled generation prompting—Instructing the LLM to admit lack of knowl- edge when information is insufficient. – Few-shot prompting—Providing examples to guide the LLM's response format or style. – Chain-of-thought (CoT) prompting—Introducing intermediate reasoning steps for complex tasks. – Advanced techniques—These include self-consistency, generated knowledge prompting, and tree of thought.

- The choice of augmentation technique depends on the task complexity, desired output format, and LLM capabilities.

Generation

- Generation is the final step in which the LLM produces the response based on the augmented prompt.

- LLMs can be categorized based on how they've been trained, how they can be accessed, and the number of parameters they have.

- Supervised fine-tuning, or SFT, improves context use and domain optimization, enhances coherence, and enables source attribution; however, it comes with challenges such as cost, risk of overreliance on retrieval, and potential tradeoffs with inherent LLM abilities.

- The choice between open source and proprietary LLMs depends on customization needs, long-term costs, and data sensitivity.

- Larger models come with superior reasoning, language understanding, and broader knowledge, and generate more coherent and contextually accurate responses but come with high computational and resource requirements.

- Smaller models allow faster inference, lower resource usage, and are easier to deploy on edge devices or resource-constrained environments but do not have the same language understanding abilities as large models.

- Popular LLMs include offerings from OpenAI, Anthropic, Google, and similar, and open source models are available through platforms such as Hugging Face.

- The choice of LLM depends on factors such as performance requirements, resource constraints, deployment environment, and data sensitivity.

- The choice of LLM for RAG systems requires careful consideration, experimentation, and potential adaptation based on performance.

Chapter 5: RAG Evaluation Accuracy, Relevance and Faithfullness

This Chapter Covers

- The need and requirements for evaluating RAG pipelines
- Metrics, frameworks, and benchmarks for RAG evaluation
- Current limitations and future course of RAG evaluation

Key Aspects of RAG Evaluation

Quality scores

Required abilities

Evaluation metrics

RETRIEVAL METRICS Accuracy Precision Precision@k Recall F1-score Mean Reciprocal Rank (MRR) Mean Average Precision (MAP) Normalized Discounted Cumulative Gain (nDCG)

RAG-SPECIFIC METRICS

The three quality scores used to evaluate RAG applications are:

Context Relevance Answer Faithfulness Answer Relevance

These scores specifically answer the following three questions:

- Is the information retrieval relevant to the user query?
- Is the generated answer rooted in the retrieved information?
- Is the generated answer relevant to the user query?

CONTEXT RELEVANCE

Context Relevance = relevant retrieved / total retrieved

ANSWER FAITHFULNESS

Answer Faithfulness = clains in context / claims in answer

Coverage = relevant context clains in response / relevant context claims

ANSWER RELEVANCE

Answer Relevance = 1/N x SUM(i=1 to N) [Similaritry(Query, LLM-Query[i])]

Human evaluations and ground truth data

Frameworks

Frameworks provide a structured approach to RAG evaluations. They can be used to automate the evaluation process. Can assist in the synthetic ground truth data generation.

Two Popular Frameworks:

RAGAs (Retrieval-Augmented Generation Assessment)

ARES (Automated RAG Evaluation System)

RAGAs (Retrieval-Augmented Generation Assessment)

- Synthetically generate a test dataset that can be used to evaluate a RAG pipeline.
- Use metrics to measure the performance of the pipeline.
- Monitor the quality of the application in production.
- RAGAs Uses **LLM-as-a-judge** approach
- This relies heavily on the quality of the LLM **SYNTHETIC TEST DATASET GENERATION (GROUND TRUTHS)**

RECREATING THE RAG PIPELINE

EVALUATIONS

ARES

- ARES, is a framework developed by researchers at Stanford University and Databricks.
- Like RAGAs, ARES uses an LLM as a judge approach for evaluations.

ARES requires more data than RAGAs. To use ARES, you need the following three datasets:

- **In-domain passage set**: This is a collection of passages relevant to the specific domain being evaluated. The passages should be suitable for generating queries and answers. In our case, it will be the documents that we created from the Wikipedia article.

- Human preference validation set—A minimum of approximately 150 annotated data points is required. This set is used to validate the preferences of human annotators regarding the relevance of the generated query-passage–answer triples.

- Few-shot examples—At least five examples of in-domain queries and answers are needed. These examples help prompt the LLMs during the synthetic data generation process.

Benchmarks

RGB

- Retrieval-augmented Generation Benchmark (December 2023)

RGB focuses on the following metrics for evaluation:

- Accuracy—Used for noise robustness and information integration. It is based on the exact matching of the generated text with the correct answer.

- Rejection rate—Used for negative rejection. It is measured by exact matching of the model's output with a specific rejection phrase. The rejection rate is also evaluated using ChatGPT to determine whether the responses contain rejection information.

- Error detection rate —Used for counterfactual robustness. It is measured by exact matching of the model's output with a specific error-detection phrase and is also evaluated using ChatGPT.

- Error correction rate—Used for counterfactual robustness. It measures whether the model can provide the correct answer after identifying errors

Multi-hop RAG

Inference—Synthesizing information across multiple sources (e.g., Which report discusses the supply chain risk of Apple—the 2019 annual report or the 2020 annual report?)

- Comparison—Comparing facts from different sources (e.g., Did Netflix or Google report higher revenue for the year 2023?)

- Temporal—Analyzing the temporal ordering of events (e.g., e.g. Did Apple introduce the AirTag tracking device before or after the launch of the 5th generation iPad Pro?)

- Null—Queries not answerable from the knowledge base

CRAG

COMPREHENSIVE RAG BENCHMARK (Meta & HKUST)

For each question in the evaluation set, CRAG labels the answer with one of four classes:

- Perfect: The response correctly answers the user's question and contains no hal- lucinated content (scored as +1).
- Acceptable: The response provides a useful answer to the user's question but may contain minor errors that do not harm the usefulness of the answer (scored as +0.5).
- Missing: The response is "I don't know", "I'm sorry I can't find ...", a system error such as an empty response, or a request from the system to clarify the original question (scored as 0).
- Incorrect: The response provides wrong or irrelevant information to answer the user's question (scored as −1).

Limitations and Best Practices

Lack of standardized metrics

- Precision, recall, and F1-score do not fully capture the nuances of generative responses
- Generation metrics such as BLEU, ROUGE do not fully capture the context awareness for RAG
- RAG-specific metrics such as answer relevance, context relevance, and faithfulness bring the necessary nuances for RAG evaluation but lack calculation standardization
- Best practice: Compare the results on RAG specific metrics from different frameworks.

Overreliance on LLM as a judge

- Subjectivity issues
- Lack of domain or case specific knowledge
- Issue with self-reference (using same LLM)
- Try to use different generation & evaluation LLMs
- Best practice:
    - Focus on use-case-specific metrics to assess quality, coherence, usefulness
    - Incorporate human judgements in your workflow

Lack of Use Case Subjectivity

- Frameworks have a generalized approach to evaluation

- Best practice: Use a benchmark that is tailored to your domain.

Benchmarks are Static

- Most benchmarks are static and do not account for the evolving nature of information.
- Best practice: Employ careful sampling of test cases for evaluation

Scalability and cost

- Evaluating large-scale RAG systems is more complex than evaluating basic RAG pipelines.

- It requires significant computational resources.

- Benchmarks and frameworks do not generally account for metrics such as latency and efficiency,

- Best practice:

    - Employ careful sampling of test cases for evaluation.
    - Incorporate workflows to measure latency and efficiency.

Summary

RAG evaluation fundamentals ¡ RAG evaluation assesses how well systems reduce hallucinations and ground responses in the provided context. ¡ Three key quality scores for RAG evaluation are context relevance, answer faith- fulness, and answer relevance. ¡ Four critical abilities required of RAG systems include noise robustness, negative rejection, information integration, and counterfactual robustness. ¡ Additional considerations include latency, robustness, bias, and toxicity of responses. ¡ Custom use-case-specific metrics should be developed to evaluate performance.

Evaluation metrics ¡ Retrieval metrics include precision, recall, F1-score, mean reciprocal rank (MRR), mean average precision (MAP), and normalized discounted cumulative gain (nDCG). ¡ Accuracy, precision, recall, and F1-score do not consider the ranking order of the results. ¡ RAG-specific metrics focus on context relevance, answer faithfulness, and answer relevance. ¡ Human evaluations and ground truth data play a crucial role in RAG assessment.

Evaluation frameworks ¡ Frameworks such as RAGAs and ARES automate the evaluation process and assist in synthetic data generation. ¡ RAGAs is an easy-to-implement framework that can be used for quick evaluation of RAG pipelines. ¡ ARES uses a more complex approach, including classifier training and confi- dence interval calculations.

Benchmarks ¡ Benchmarks provide standardized datasets and metrics for comparing different RAG implementations on specific tasks. ¡ Popular benchmarks such as SQuAD, natural questions, HotpotQA, and BEIR focus on retrieval quality. ¡ Recent benchmarks such as RGB, multi-hop RAG, and CRAG are more holistic from a RAG perspective. ¡ Benchmarks focus on different aspects of RAG performance, such as multi-hop reasoning or specific domains.

Limitations and best practices ¡ Challenges in RAG evaluation include lack of standardized metrics, overreliance on LLMs as judges, and static nature of benchmarks. ¡ Best practices include using multiple

frameworks, incorporating use-case-specific metrics, and regularly updating evaluation data. ¡ Balancing automated metrics with human judgment and considering use-case- specific requirements is crucial. ¡ The field of RAG evaluation is evolving, with new frameworks and benchmarks constantly emerging. ¡ Developers should stay informed about new developments and adapt their evalu- ation strategies accordingly.

Part 3: RAG in Production